WeGo-Robotics

HACK

PIPER Studio으로 치르는 로봇 AI 해커톤

01 왜 PIPER Studio인가? 쉽고 빠르게!

02 원만한 운용을 위한 팁

03 앞으로 나아갈 방향!!!

← 메뉴

ⓒ 2026 WeGo-Robotics. 본 자료의 무단 복사·배포·유출은 법적으로 보호받지 못하며, 관련 법령에 따라 책임이 따를 수 있습니다.

실제 치러진 로봇 AI 해커톤 — 사례

2025~2026 국내외 사례 4건 · 공통점: **저가 로봇팔** + LeRobot + **모방학습**, 30~100시간, **실물 시연으로 평가**

🤗 LeRobot Worldwide Hackathon

2025.6.14~15 · Hugging Face

3,000명+ · 44개국 · 100+ 지역 동시 개최 · SO-100/101 팔 · ACT · Diffusion · SmolVLA · GR00T

글로벌 2위: 3명 · 주말 · 3D 프린팅 SO-100으로 **셔츠 접기 + 약 분류** · 2026년에도 시애틀(48h)·뮌헨(SO-101 50대+)·LA·파리(OpenArm) 개최

📍 서울 노드 (SK mySUNI)

2025.6 · ~50팀

글로벌 Top 8 — 색 기준 pick & place · **ACT · 50 ep · 60k step · 성공률 ~90%**

Team 112 — 서랍 열고 사탕 옮기기: **오후 1시 성공 → 5시 같은 셋업에서 실패** (장비·소프트웨어 상태 변화 의심) · 다른 팀: GR00T 실험은 시간 내 결과 확보에 어려움

🏫 한성대 Physical AI 해커톤

2026.2.7~8 · 1박 2일

80명 · 20팀(2~4인) · SO-ARM 100/101 **양팔** · LeRobot · 협력 로보시지

미션 3종: 3색 분류 · 정밀 포장 · 세밀 접기 → 1일차 안전교육·개발, 2일차 발표·심사·결과물 업로드

🎓 KAIST·서울대 100시간 로봇 해커톤

2026.8.3~8 · RoboticUS

10팀 30명 · 교육 2일(전원·관절 제어) + 미션 3일 · 엔젤로보틱스 phact 구동기 + Jetson AGX

심사: 미션 달성도 · 기술 활용 · 창의성 · 발표·시연 — 교수 30% · 참가자 상호 30% · 후원사 30% · 시민 10%

작은 과제도 완성도 높게
색상별 집기·옮기기 성공률 약 90%
글로벌 Top 8 기록

장비 상태를 끝까지 유지
성공하던 동작이 4시간 뒤 실패
장비·소프트웨어 상태 변화 의심

학습 시간에 맞춰 모델 선택
대형 모델에 도전했지만
GPU 이용 시간 부족으로 학습 미완료

기초 교육부터 실습까지
행사 중 교육 진행 (KAIST 2일 · 한성대 반나절)
이 자료로 데이터 수집·모델 학습·디버깅 실습을 준비

출처: huggingface.co/LeRobot-worldwide-hackathon · lightning.ai 우승팀 인터뷰 · sudormrf.run 서울 참가 후기 · 뉴데일리 2026.2.10 · KAIST 뉴스 2026.8.4

← 메뉴

WeGo-Robotics

실제 해커톤에서 드러난 문제 4가지

2025 LeRobot Worldwide Hackathon 후기 · LeRobot 공식 사용 절차 참고

01 LeRobot 설치·설정이 복잡

실행 환경·장치·명령어 설정 필요
버전·지원 기능에 따라 실험에 제약

공식 설치 문서 ↗ · 서울 운영진 후기 ↗

02 카메라 문제로 데이터 손실

행사 막판 시연 데이터 36개 손실
수집한 데이터의 상태 확인이 중요

셔츠 접기 Team 250 · 참가팀 인터뷰 ↗

03 GPU를 확보해도 학습 중단

Hub 장애 후 Colab으로 전환
A100을 사용한 야간 학습도 중단

에인트호번 참가팀 · 참가자 후기 ↗

04 실행 조건 변화에 취약

서울: 4시간 뒤 같은 동작 재현 실패
뮌헨: 카메라 이동 후 동작 실패

서울 운영진 후기 ↗ · 뮌헨 참가팀 후기 ↗

준비할 것 · 환경 사전 구성 / 녹화 검수 / 학습 복구 / 실행 조건 유지

← 메뉴

WeGo-Robotics

WeGo-Robotics

WHY PIPER STUDIO

왜 PIPER Studio인가?

쉽고 빠르게 — 앞의 네 가지 문제를 **화면 뒤에서** 처리한다

1. 편의성 — 설치 한 줄 · CLI 대신 화면
2. 카메라 운용 — 프로파일 · 조명 계측과 알람
3. 에피소드 확인 — 녹화하고 바로 검수
4. 실험 설계 확인 — 엔코더 프로브 · AprilTag 정렬 검사

이어서 권장 사양 · 화면 지도 · 페이지별 안내

← 메뉴

① 편의성 — 스크립트 하나로 설치

해커톤 전날 팀 PC마다 한 번 · 이미 돼 있으면 건너뛰고 두 번 돌려도 같다

```
curl -fsSLO https://raw.githubusercontent.com/WeGo-Robotics/wego-piper-web/master/deploy/piper-install.sh
chmod +x piper-install.sh
./piper-install.sh              # 최신   |  ./piper-install.sh --check  → 상태만 확인
```

스크립트가 하는 일 · 멈추는 곳

1. 도커 확인 → 없으면 설치 명령 찍고 멈춤
2. 이미지 pull (두 번째부터 바뀐 레이어 ~100MB)
3. 호스트용 파일 꺼냄 — 데몬 · udev · compose
4. 전제 확인 → udev → 데몬 유닛 → 컨테이너

설치 후 남는 일 — 전부 화면에서

CAN 어댑터 이름
로봇 페이지에서 어느 어댑터가 리더·팔로워인지 지정하고 프리셋으로 저장 — 재부팅해도 유지

카메라 등록 · 프로파일
카메라 페이지에서 등록 → 회색 카드 보정 → 프로파일 활성

HF 토큰
설정 → 저장소 탭. 호스트 로그인은 컨테이너가 못 본다

전날 30분 — 스크립트 실행 → 대시보드 서비스 전부 녹색 확인 → 팔 · 카메라 등록. 당일 아침엔 켜기만

전날 미리: 이미지가 아직 레지스트리에 없으면 `PIPER_IMAGE=<레지스트리>/piper-web-backend ./piper-install.sh` 로 주소를 직접 넘긴다 · 자세한 표는 README "설치"

← 메뉴

WeGo-Robotics

① 편의성 - LeRobot은 CLI · PIPER Studio는 화면

같은 LeRobot이다 — 차이는 **배우는 데 걸리는 시간**뿐

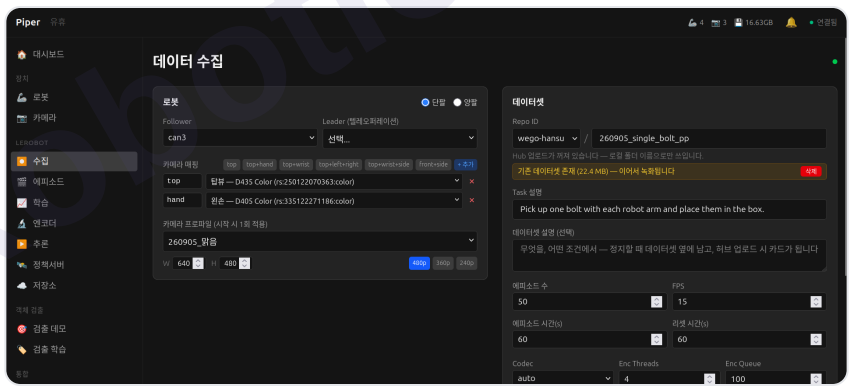
LeRobot 그대로 — 터미널

```
$ python -m lerobot.record \
  --robot.type=piper --robot.port=can0 \
  --teleop.type=piper_leader --teleop.port=can1 \
  --robot.cameras='{"top": {...}, "hand": {...}}' \
  --dataset.repo_id=team/cube_in_box \
  --dataset.num_episodes=50 --dataset.fps=30 \
  --dataset.single_task="Pick the cube ..."

ValueError: camera index 4 not found
$ _
```

- 명령어 · 인자 이름 · 경로 · 환경 설치를 **먼저 배워야** 한 번 돈다
- 오타 하나 = 처음부터 · 터미널 닫히면 학습도 끝
- 처음 하는 사람: **하루**

PIPER Studio — 브라우저



- 같은 명령을 **드롭다운과 버튼**으로 · 인자는 화면이 대신 조립
- 설정은 저장되고 · 학습은 브라우저 닫아도 돈다
- 처음 하는 사람: **30분**

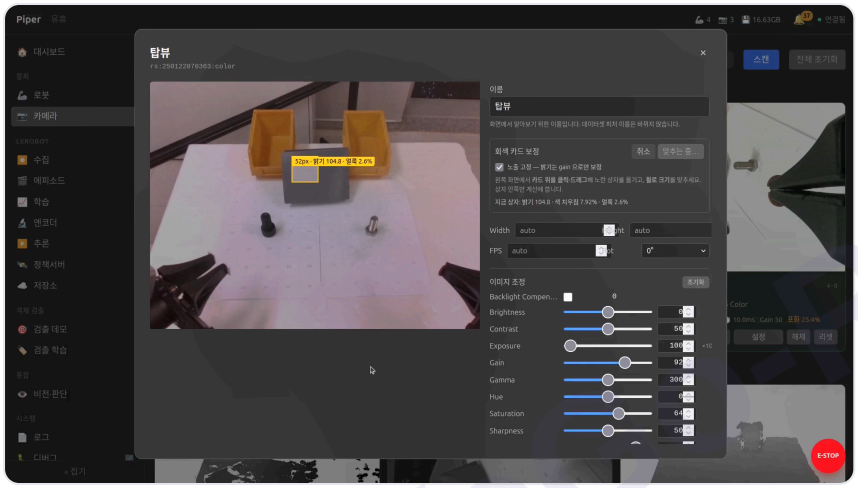
배우는 데 쓰던 시간이 **데이터 모으는 시간**이 된다

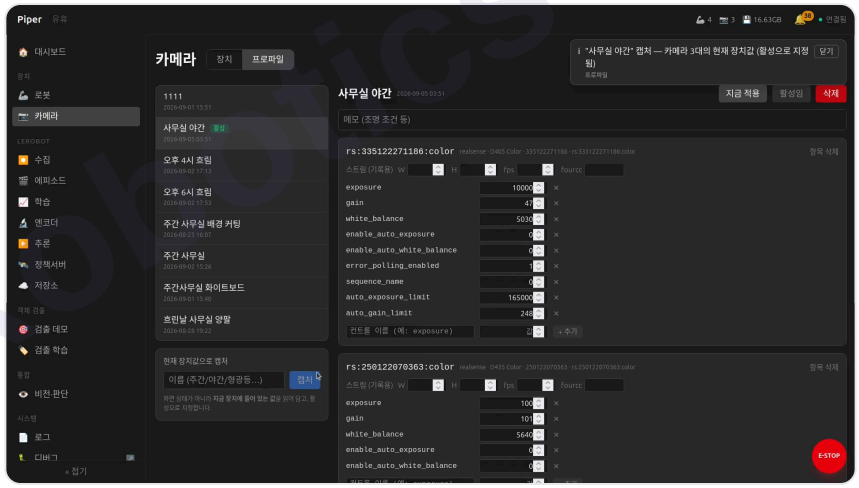
← 메뉴

WeGo-Robotics

② 카메라 운용 - 픽셀을 어제와 같게

탭: 장치 · 프로파일 — 수집 때와 추론 때 픽셀이 같아야 한다. 그걸 **사람 감이 아니라 저장된 값**으로





회색 카드 보정
카드 두고 [보정 시작] → 노출 잠그고 WB·밝기 자동 보정

수동 고정값
Exposure · Gain · WB 자동 off → 프로파일로 저장

이름 붙여 캡처
조명 조건별로 카메라 전부의 현재 값을 한 벌로 저장

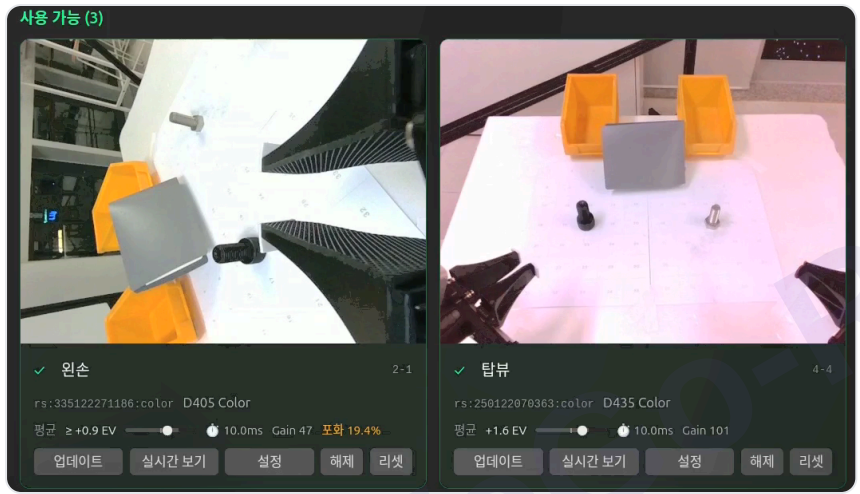
골라서 적용
[지금 적용] 즉시 · [활성] 지정하면 연결할 때마다 자동

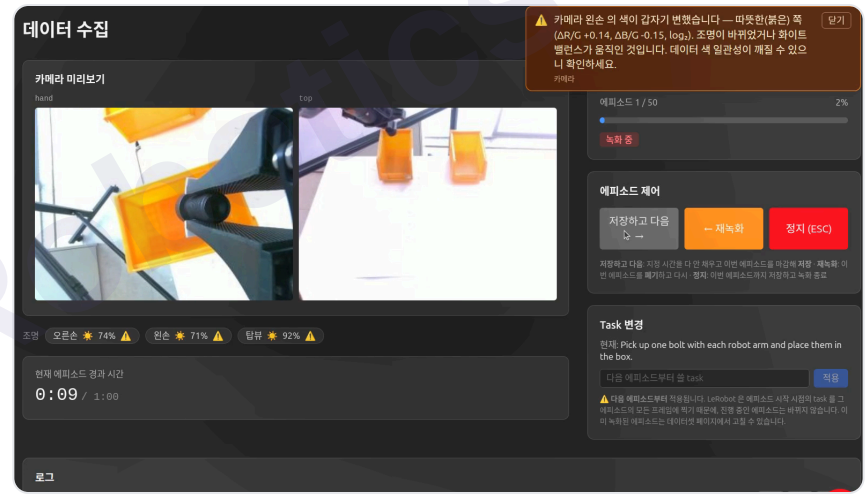
핵심 — 카메라 값을 사람 감이 아니라 프로파일로 · 재부팅·둘째 날에도 어제 픽셀 그대로

← 메뉴

② 카메라 운용 - 조명 계측, 수치로 비교하고 변하면 알람

"어제보다 어두운 것 같은데"를 감이 아니라 **숫자**로 · 수집·추론 중 조건이 바뀌면 화면이 먼저 안다





밝기를 숫자로
카메라마다 EV · 노출 · Gain · 포화 % 표시

비교가 된다
카메라끼리 · 어제와 · 프로파일 전후 — 같은 자로 잰 값

작업 중 실시간 감시
수집·추론 화면에 카메라별 조명 칩 · 녹화 중에도 갱신

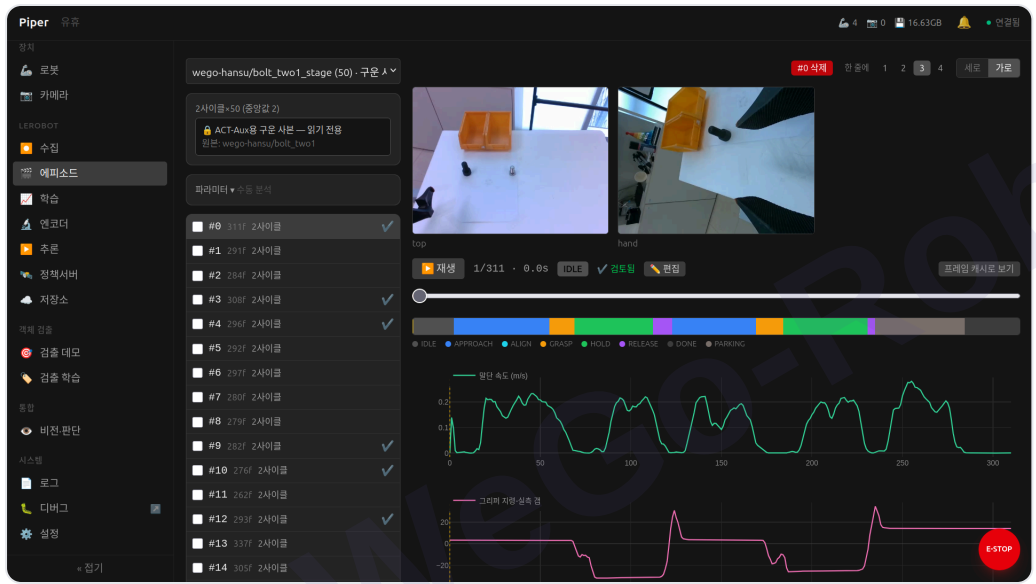
변하면 알람
밝기·색이 급변하면 토스트 + 벨 — "왼손 색이 붉은 쪽으로 변함"

알람 뜨면 — 수집 중단 → 원인(창문·스위치·WB 자동) 복구 → 칩 정상 확인 → 재개. 알람 무시하고 찍은 에피소드는 삭제

← 메뉴

③ 에피소드 확인 — 녹화하고 바로 검수

재생 + 관절 신호 + 작업 단계(페이즈) 한 화면 · 검수가 빨라지면 **나쁜 데이터가 학습에 들어갈 틈이 없다**



재생 + 그래프 동기
스크러버 움직이면 영상·말단 속도·그리퍼 갭이 같이 — 멈칫 = 평평한 구간

페이즈 트랙 자동 라벨
APPROACH → GRASP → HOLD → RELEASE → DONE · 순서 어긋나면 대본 위반

검토 ✓ 표시
본 에피소드는 체크 · 10 ep마다 PM이 훑는다

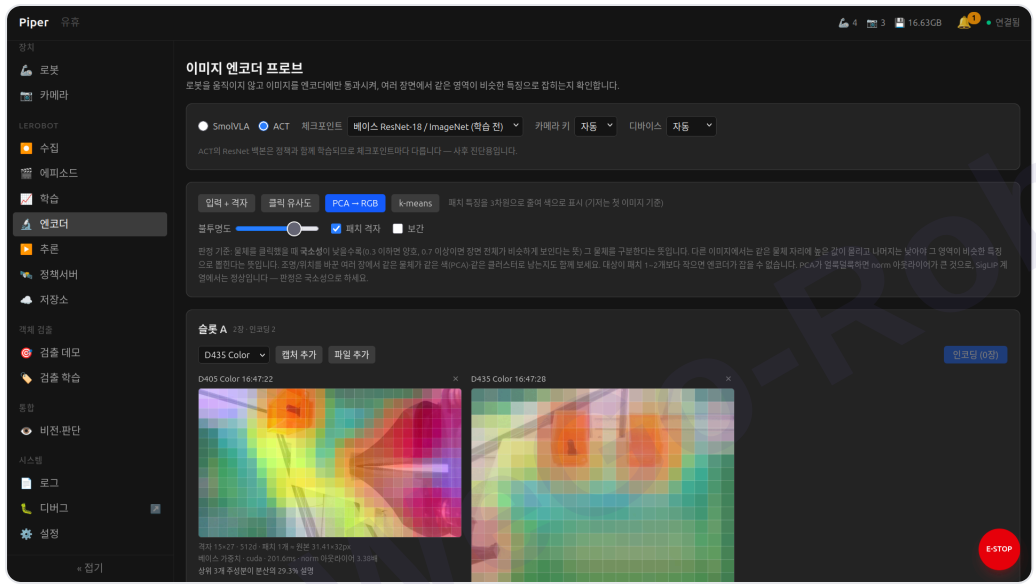
에피소드 삭제
불량이면 우상단 [#N **삭제**] 버튼 한 번 · 여러 개는 체크박스로 골라 일괄 · 번호 꼬임 없음

검수 루틴 — 페이즈 순서 → 평평한 구간·튀는 점 → hand에 손 → 삭제 후 재녹화

← 메뉴

④ 실험 설계 확인 — 인코더가 뭘 보는지 눈으로

로봇 안 움직이고 이미지만 인코더에 통과 · **앞 슬라이드의 "못 보는 것"을 과제 확정 전에 직접 확인**



로봇 없이 인코더만
SmolVLA / ACT 골라 이미지 통과 · 패치 특징을 색으로 표시

판정 — 국소성
물체 클릭 → 0.3 이하 양호 · 0.7 이상이면 장면 전체가 비슷해 보임

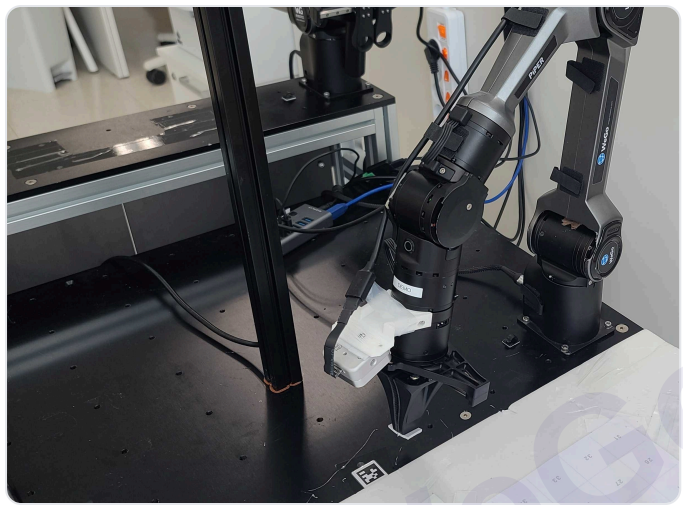
너무 작으면 못 본다
패치 1개 ≈ 31×32px · 물체가 패치 1~2개면 손목 카메라 몫

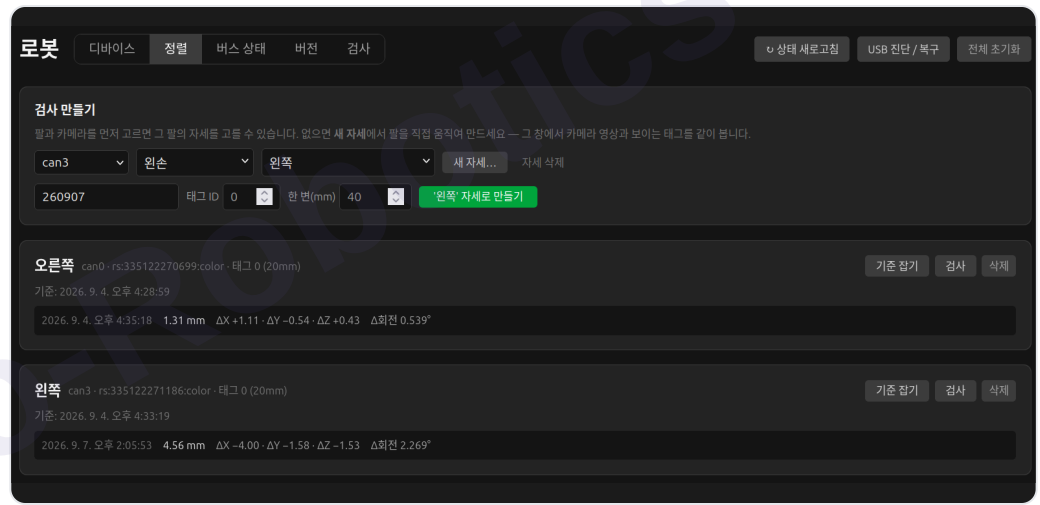
사용법 — 물체를 그리드 9칸에 놓고 9장 캡처 → 전부 같은 색이면 OK · 흩어지면 물체 크기 ↑ · 배경 대비 ↑ · 카메라 가까이

← 메뉴

④ 실험 설계 확인 — AprilTag로 팔 정렬 검사

관절이 밀리면 팔은 자기가 틀어진 걸 모른다 — **밖에서 재야 안다** · 로봇 페이지 정렬 탭





손목 카메라 + 작업대 태그
정해진 자세로 보내고 태그가 화면 어디에 보이나

기준 대비 차이만
정상일 때 한 번 찍은 기준과 비교 · 핸드아이 보정 불필요

검사 만들기 → 기준 잡기
팔 · 카메라 · 자세 · 태그 ID · 한 변(mm) 정하고 셋업 직후 [기준 잡기] 한 번

검사 결과 읽기
오른쪽 1.31 mm 정상 · 왼쪽 4.56 mm · **회전** 2.3° = 3일 사이 틀어짐 → 원인 찾고 그 뒤 데이터 의심

언제 — 셋업 직후 기준 → 20 ep마다 · 충돌·과부하 후 · 추론 실패가 갑자기 늘 때. 재칼리브레이션 전에 먼저 이걸로 확인

← 메뉴

권장 사양 — 팀 PC 한 대

설치 스크립트가 아래 항목을 확인하고 **미달이면 필요한 명령을 찍고 멈춘다** · 전날 `./piper-install.sh --check` 로 미리 확인

항목	최소	권장
OS	Ubuntu 22.04	Ubuntu 22.04 / 24.04 · 사용자가 docker 그룹
GPU	컴퓨트 능력 **7.5 이상** — RTX 20xx · T4 (Pascal·Volta 불가)	RTX 3090 / 4090 · VRAM 24GB — SmolVLA 파인튜닝까지 여유
드라이버	CUDA 13.0 이상 (nvidia-driver-580) + nvidia-container-toolkit	같음 · 설치 후 재부팅 확인
런타임	docker + compose v2 · python3-venv · redis-server	같음 — 나머지는 이미지 안에 전부
디스크	50GB 여유	**100GB+ SSD** — 데이터셋 수 GB + 체크포인트 ~200MB × 10개/학습
USB	USB 3.0 포트 2 (카메라) + USB-CAN 2	카메라는 **서로 다른 컨트롤러**에 직결 · 허브 ✗
네트워크	없어도 동작 (localhost)	팀원 노트북이 같은 LAN에서 접속 · HF 업로드용 인터넷

GPU가 없거나 약하면
학습은 SSH 원격 GPU · 추론은 정책서버(다른 PC) — 로봇 PC는 카메라·CAN만 담당해도 됨

VRAM 8~12GB면
ACT만 · batch 8 유지 · SmolVLA는 포기하거나 원격

안 되는 것
GTX 10xx · Pascal/Volta 서버 GPU · Windows/WSL · macOS — 이미지의 torch가 sm_75 이상만 포함

← 메뉴

WeGo-Robotics

WeGo-Robotics

화면 지도 — 페이지 ↔ 해커톤 단계

브라우저 좌측 메뉴 그대로 · 굵은 것이 해커톤 필수 화면 · 나머지는 필요할 때만

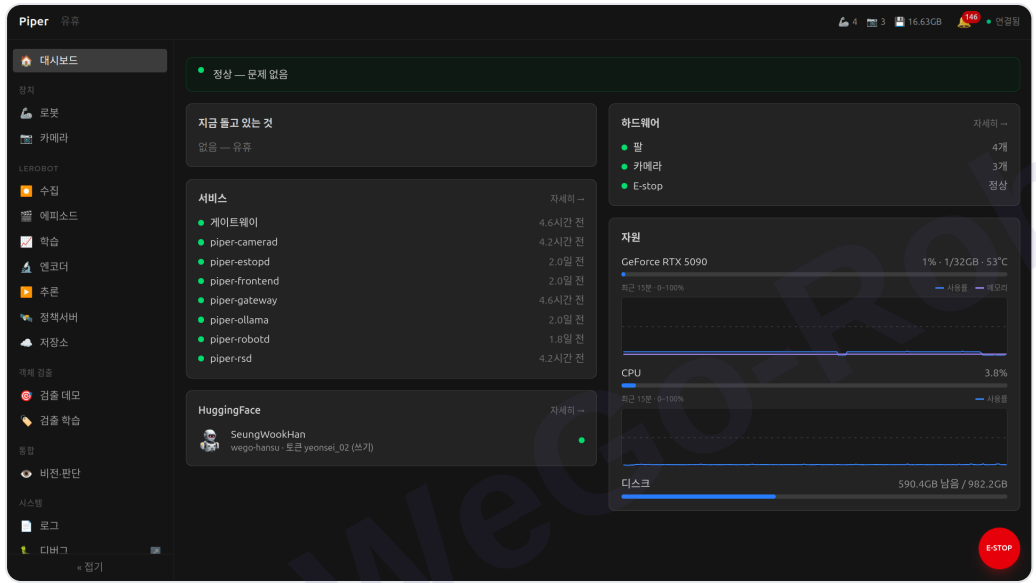
단계	페이지	하는 일	담당
준비	**대시보드**	장치·데몬·GPU·HF 로그인·E-stop 상태 한눈에 · 지금 돌고 있는 활동	전원
준비	**로봇**	팔 등록·CAN 프리셋 · 마스터/슬레이브 판별 · 토크 ON/OFF · 파킹 · 조그 · 정렬·검사 탭	셋업 담당
준비	**카메라**	카메라 등록 · 노출·WB 프로파일 · 회색 카드 보정 · 조명 변화 감시	셋업 담당
준비	설정	일반 · 안전 · 서비스(데몬 재시작) · 저장소(HF 계정·토큰 · 모델 검색 경로)	인프라
수집	**수집**	텔레옵 · 에피소드 녹화 · 카메라 매핑(top·hand) · task 문장 · 설정 저장 · 이어서 녹화	오퍼레이터 + 셋업
검수	**에피소드**	뷰어(재생·관절 신호·페이즈 트랙) · 삭제 · task 수정 · split/merge	PM
학습	**학습**	데이터셋·정책 선택 · 프리셋 · 로컬/원격 GPU · 진행률·loss · 브라우저 닫아도 계속	인프라
진단	엔코더	이미지 엔코더 프로브 — ACT/SmolVLA 인코더가 물체를 구분하는지 로봇 없이 확인	PM
학습·공유	저장소	모델/데이터셋 × 로컬/Hub · 체크포인트 목록·추론 시작 · smolvla_base 다운로드 · 업로드	인프라
평가	**추론**	체크포인트 배포 · 로컬/서버 모드 · 실시간 파라미터 · 액션 필터 · 일시정지·수동 조작 · 평가 기록	평가 담당 + E-stop
평가	정책서버	다른 PC의 GPU로 추론 (gRPC :8088) — 로봇 PC에 GPU 없을 때	인프라
분석	**로그 · 디버그**	추론 CSV(목표·필터·실제 관절) 다운로드·차트 · 디버그 뷰어(프레임 스크러버·필터 시뮬)	PM
확장	검출 데모 · 검출 학습 · 비전·판단	YOLO 검출 · 커스텀 YOLO 학습 · YOLO→LLM→VLA 분리수거 루프 — ★★★ 팀용	—

한 PC에 브라우저 여러 개 접속 가능 → 오퍼레이터는 수집, PM은 에피소드, 인프라는 학습 페이지를 **동시에** 본다

← 메뉴

대시보드 — 접속하면 보이는 첫 화면

장치 · 서비스 · GPU · HF 로그인이 한 화면 · 여기가 녹색이면 셋업의 절반은 끝난 것



좌측 메뉴 = 워크플로우 순서
장치(로봇·카메라) → LeRobot(수집·에피소드·학습·엔코더·추론·정책서버·저장소) → 객체 검출 → 통합(비전·판단) → 시스템(로그·디버그·설정)

지금 돌고 있는 것
녹화·학습·추론 중 무엇이 GPU를 쥐고 있나. 409 뜨면 여기부터

서비스
게이트웨이 · camerad · estopd · robotd · rsd · ollama — 전부 녹색이어야. 하나라도 빨간색이면 설정 → 서비스에서 재시작

하드웨어 · 자원
팔 수 · 카메라 수 · E-stop 상태 · GPU 사용률/VRAM/온도 · 디스크 잔량

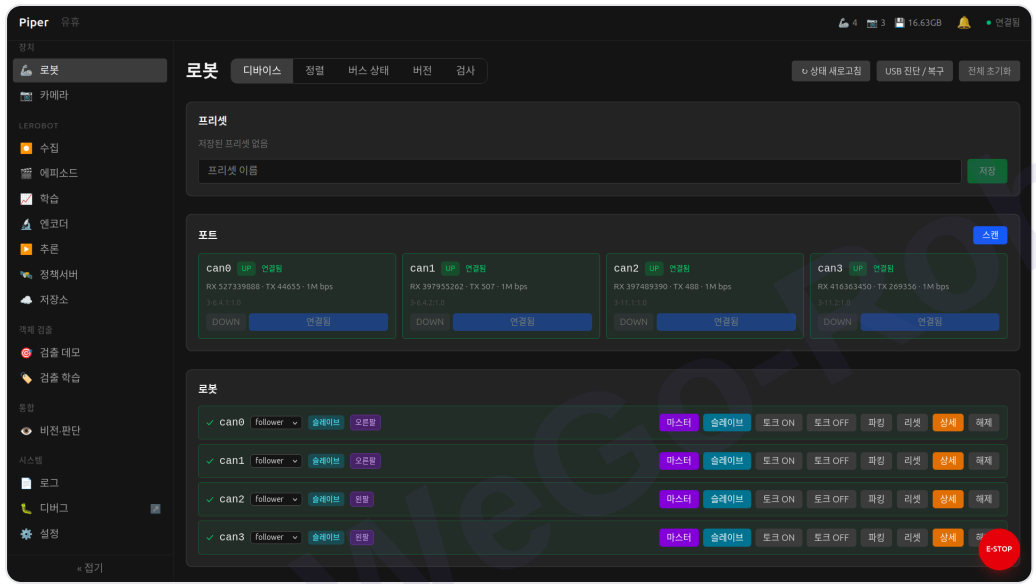
HuggingFace
계정 · 토큰 권한(쓰기) 배지. 미로그인이면 데이터셋 업로드·모델 푸시 불가

E-STOP 버튼 — 모든 화면 우하단에 항상. 상단바 알림 벨은 장치 소멸·조명 급변 경보

← 메뉴

로봇 페이지 — 팔을 등록하고 믿을 수 있게

터미널의 `find_port` · `calibrate` · `teleoperate` 가 여기 한 화면 · 탭: 디바이스 · 정렬 · 버스 상태 · 버전 · 검사



프리셋
팔·CAN 배정을 이름 붙여 저장. 재부팅 후 한 번에 복원

포트
can0~3 UP/DOWN · RX/TX · 1 Mbps. 스캔으로 새 어댑터 찾기

로봇 행
follower/leader · 마스터/슬레이브 · 토크 ON/OFF · 파킹 · 리셋 · 상세

마스터/슬레이브 판별
이동 명령 넣어 **안 움직이면 마스터(리더)**. 리더는 외부 명령을 무시 — 헷갈리면 이 버튼

토크 ON/OFF · 파킹
에피소드 시작 자세 = 파킹 자세로 통일. 전원 끄기 전 파킹 → 낙하 방지

움직이기 세 방식
리더암 릴레이(수집용) · 관절 조그(시작 자세·간섭 확인) · 엔드포인트 XYZ(도달 범위). **셋 다 팔로워만 움직인다**

정렬 탭 — AprilTag
정해진 자세에서 태그가 카메라에 어디 보이나 → 정상일 때 기준과의 차이. **관절 쉬프트를 밖에서 잡는 유일한 방법**

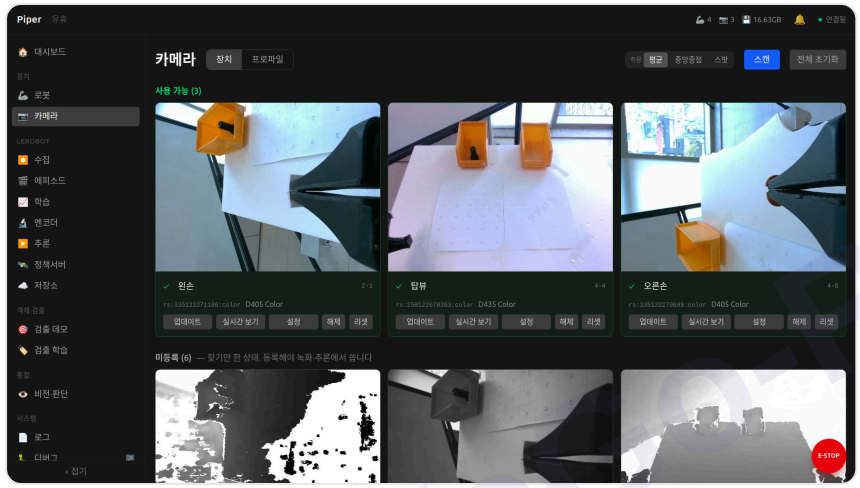
검사 탭
정해진 모션으로 추종 오차·전류·온도 로그 → 여섯 관절 중 혼자 다른 놈

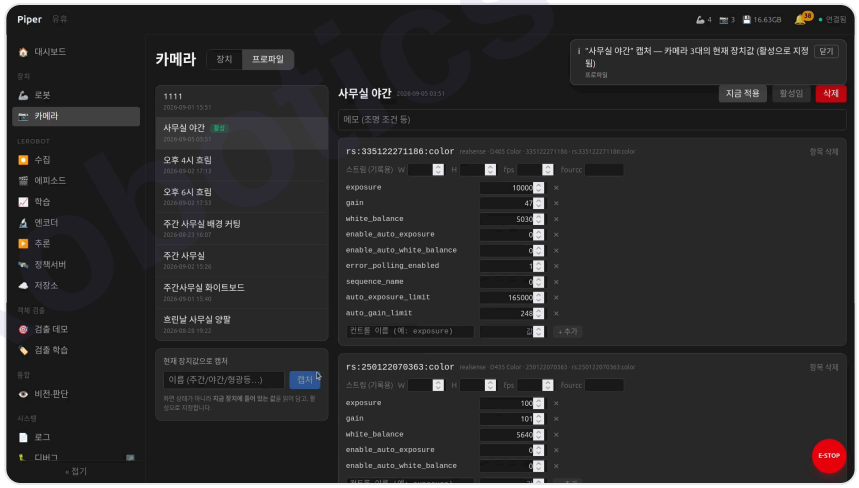
해커톤 루틴 — 셋업 직후 정렬 기준 촬영 1회 → 20 ep마다 · 실패 급증 시 검사 탭부터

← 메뉴

카메라 페이지 — 장치 · 프로파일

탭 두 개 — **장치**(등록 · 실시간 · 밝기 계측) · **프로파일**(설정값 저장 · 적용)





장치 탭 — 등록
찾기만 한 카메라는 **등록**해야 수집·추론에서 쓴다 · RealSense는 color·depth·IR 따로

카드마다 계측값
평균 EV · 노출 · Gain · 포화 % · 실시간 보기 · 설정(회색 카드 보정)

프로파일 탭 — 캡처
조명 조건별 이름으로 카메라 전부의 현재 값을 한 벌 저장

지금 적용 · 활성
[활성]이면 연결할 때마다 자동 적용 — 재부팅해도 같은 픽셀

해커톤 루틴 — 셋업 직후 회색 카드 보정 → 프로파일 캡처·활성 → 수집 페이지에서 그 프로파일 선택. 이후 값은 건드리지 않는다

← 메뉴

수집 페이지 — 설정 한 번, 그다음은 버튼만

터미널의 `lerobot.record` 인자 10개가 폼 하나 · 저장되므로 다음 에피소드부터는 시작 버튼만



로봇 — 단팔/양팔 · Follower · Leader
로봇 페이지에서 등록한 팔이 드롭다운에. Leader가 텔레옵 입력

카메라 매핑 — top · hand
데이터셋 관측 키 이름을 여기서 정한다. **체크포인트가 이 이름을 요구**하므로 한 번 정하면 끝까지. top+hand 프리셋 사용

카메라 프로파일 · 해상도
시작 시 1회 적용 · 640×480 (480p) · 두 카메라 동일

데이터셋 — Repo ID · Task 설명
HF 계정 / 이름. 기존 데이터셋 있으면 **이어서 녹화** → 보강 수집이 이 한 줄. Task 문장은 전 에피소드 동일

에피소드 수 · FPS · 시간 · 리셋 시간
50 · 30 · 20초 · 10초 권장. 리셋 시간 동안 물체 재배치 — 손 빼고 나서 다음 시작

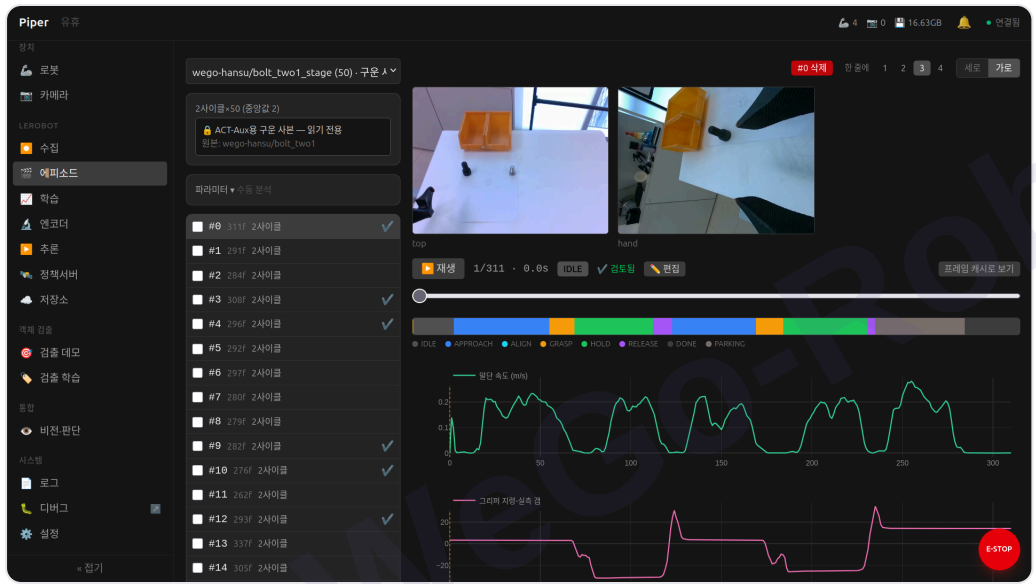
Hub 업로드 · 웹 미리보기 · 음성 안내
업로드는 수집 끝나고 켜도 됨. 음성 안내가 "시작·리셋"을 읽어줘 오퍼레이터가 화면 안 봐도 됨

⚠️ **녹화는 브라우저가 닫히면 죽는다** — heartbeat 끊김 = 안전 정지. 탭 닫기·노트북 덮기·와이파이 전환 금지 (학습과 다르다)

← 메뉴

에피소드 페이지 — 뷰어 · 편집

데이터셋 고르면 에피소드 목록 · 두 카메라 재생 · 페이즈 트랙 · 관절 신호가 한 화면



왼쪽 목록
에피소드 번호 · 프레임 수 · 검토 ✓ · 체크박스 선택

재생 · 스크러버
top · hand 영상과 아래 그래프 재생헤드가 같이 움직임

페이즈 트랙
IDLE → APPROACH → ALIGN → GRASP → HOLD → RELEASE → DONE 자동 라벨 · [편집]으로 수정

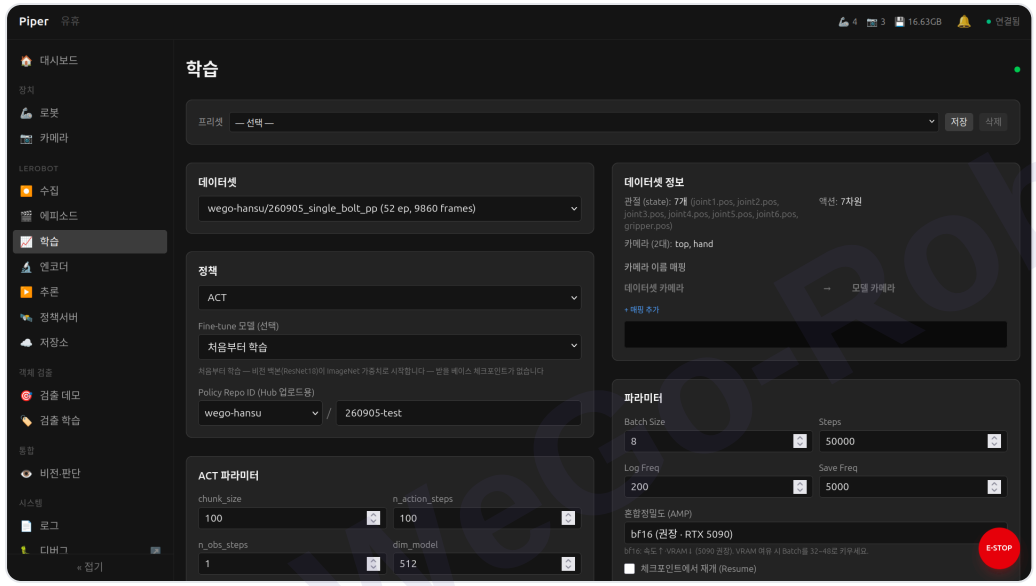
신호 그래프
말단 속도 · 그리퍼 지령-실측 갭 — 멈칫 · 미끄러짐이 보인다

삭제 · task 수정 · split/merge
[#N 삭제] 버튼 또는 체크 후 일괄 · 문장 오타 일괄 수정 · repo 합치기

← 메뉴

학습 페이지 — 기본값으로 시작, 그리고 방치

ACT 기본 설정이 ★ 과제엔 충분 · 하이퍼파라미터 탐색은 해커톤에서 최악의 시간 투자



① 데이터셋 드롭다운
"repo (52 ep, 9860 frames)" 로 뜸 — 경로 오타 없음. 오른쪽에 관절 7 · 액션 7차원 · 카메라 top/hand 자동 표시

② 정책 ACT · Fine-tune 모델
ACT는 "처음부터 학습"(ResNet-18 ImageNet 초기화). SmolVLA면 여기서 smolvla_base 선택

③ 파라미터 — 손대지 않는다
chunk_size 100 · Batch 8 · Steps 50000 · Save Freq 5000 · AMP bf16. Steps만 40000으로 줄여도 됨

④ 프리셋 저장 → 시작
"act-기본" 저장 → 보강 후 재학습 때 같은 값. Policy Repo ID 넣으면 끝나고 Hub 푸시 (토큰 사전 검사)

돌아가는 동안
systemd 유닛 — **탭 닫고 노트북 덮어도 계속**. 다시 열면 진행률·loss가 없던 동안 것까지 이어짐. 5k마다 체크포인트가 저장소에 쌓임 → 10k에서 조기 실기

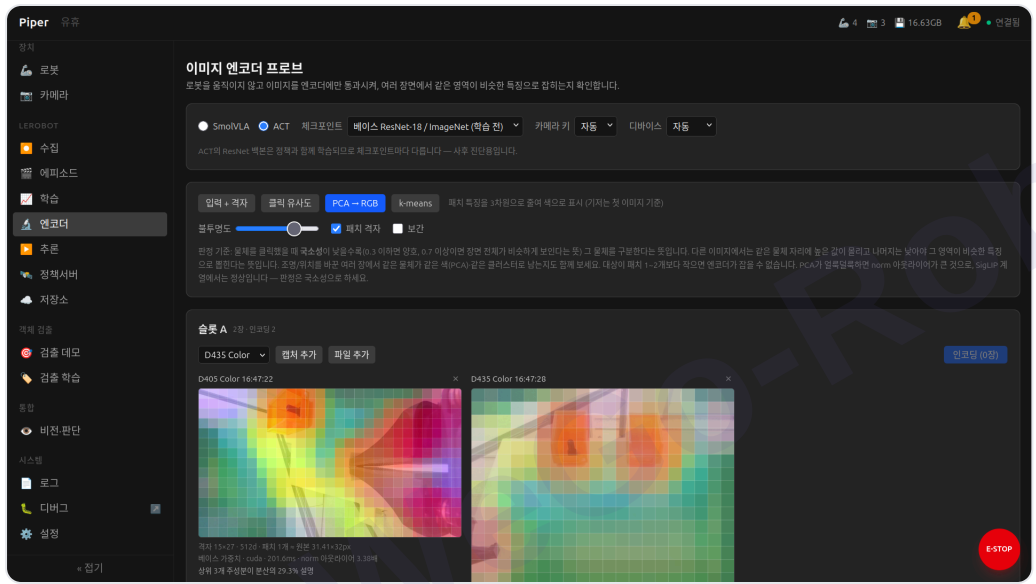
GPU 부족 → 원격
SSH 러너로 다른 머신에서 학습 · 화면은 동일. 로컬 학습 중엔 추론·녹화 409 (원격이면 안 막힘)

하지 말 것: lr·batch 튜닝 · chunk_size 변경 · 정책 2개 동시 · 학습 중 데이터셋 편집. 학습이 안 되면 답은 거의 항상 데이터 · 로그 읽기는 LeRobot 세션 ⑦

← 메뉴

엔코더 페이지 — 이미지 엔코더 프로브

로봇 없이 이미지만 인코더에 통과시켜 **정책이 물체를 구분하는지** 미리 본다



모델 선택
SmolVLA(SigLIP) / ACT(ResNet-18) · 체크포인트 · 카메라 키

보기 모드
입력+격자 · 클릭 유사도 · PCA→RGB · k-means

슬롯에 이미지 추가
카메라 캡처 또는 파일 · 여러 장을 나란히 비교

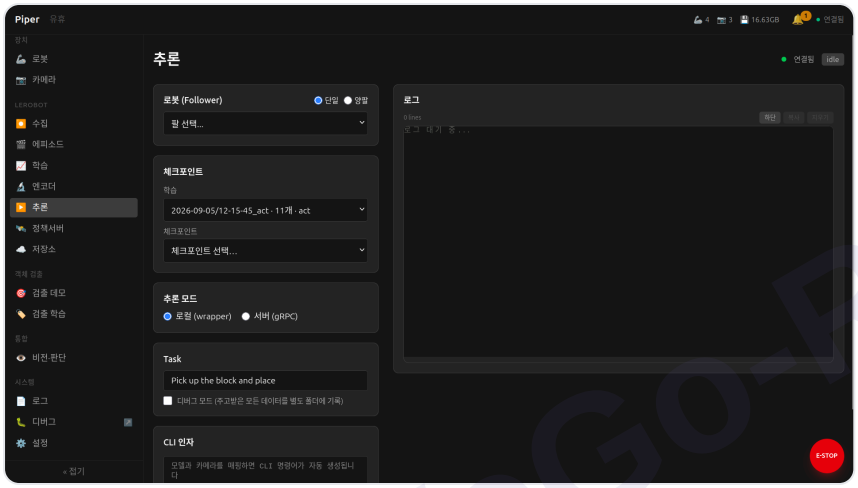
판정
국소성 0.3 이하 양호 · 같은 물체가 여러 장에서 같은 색이면 OK

언제 — 과제 물체·배경 정하기 전 · ACT vs SmolVLA 고르기 전

← 메뉴

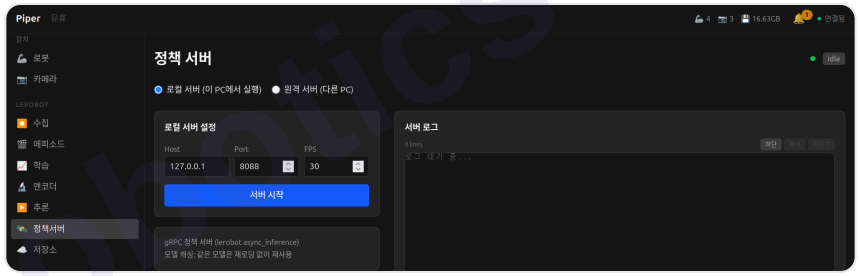
추론 · 정책서버 — 배포 · 튜닝 · 기록이 한 화면

터미널이면 파라미터 하나 바꿀 때마다 재시작 · 여기선 **돌리면서 슬라이더** · 매 스텝이 CSV로 남는다



로봇 · 체크포인트 · Task
학습 run → 체크포인트 선택 (010000 · 020000 …). Task는 학습 때와 같은 문장 — SmolVLA는 문장 바꾸면 행동 바뀜

추론 모드 로컬 / 서버(gRPC)
CLI 인자는 자동 생성. 디버그 모드 켜면 주고받은 모든 데이터가 폴더에 → 디버그 뷰어에서 재생



정책서버 — 로컬 / 원격(다른 PC)
GPU 있는 PC가 서버(:8088) · 로봇 PC가 클라이언트. 청크 100개 미리 받아 왕복 지연이 제어에 안 묻음 — 실측 19.9 fps. 체크포인트 경로는 서버 컨테이너 기준 `/data/models/…`

실행 중 실시간 파라미터
`temporal_ensemble_coeff`(떨리면 ↑) · `n_action_steps`(작을수록 반응 빠르고 진동 ↑) · 액션 필터(속도 제한+저역통과). chunk_size·모델 구조는 못 바꿈

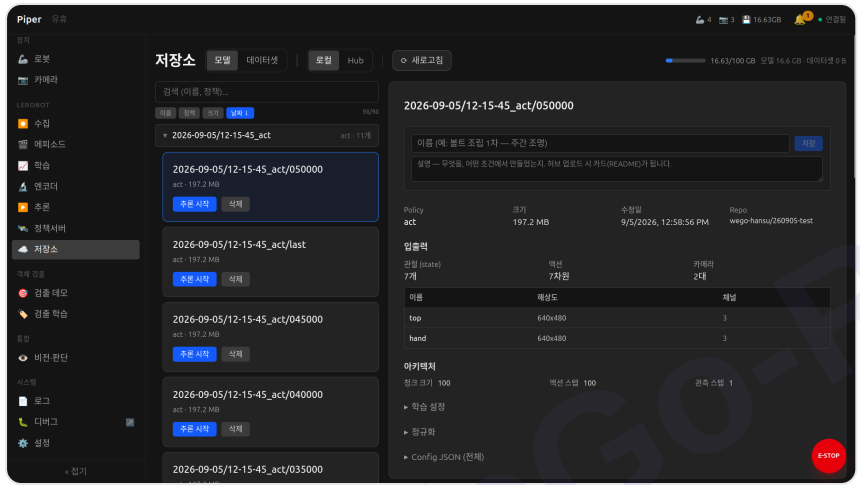
일시정지 · 수동 조작 · 평가 기록
실패 직전 멈추고 슬라이더로 팔 빼기 → 재개. 물체 리셋은 일시정지 상태에서. 시도마다 성공/실패 버튼 → 통계

추론 CSV — 매 스텝
관절마다 target · filtered · actual + fps · 큐 잔량 → 로그 페이지

← 메뉴

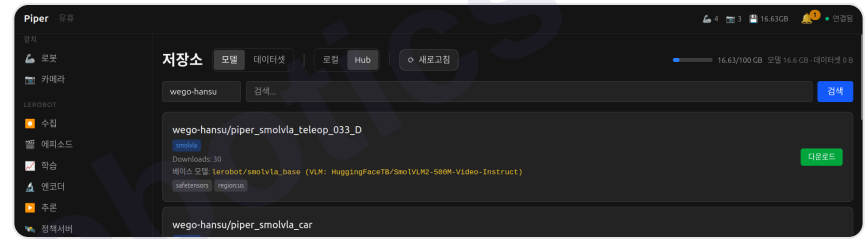
저장소 · 설정 — 체크포인트 · Hub · 경로

저장소 = 모델/데이터셋 × 로컬/Hub 한 화면 · 설정 = 일반 · 안전 · 서비스 · 저장소 탭

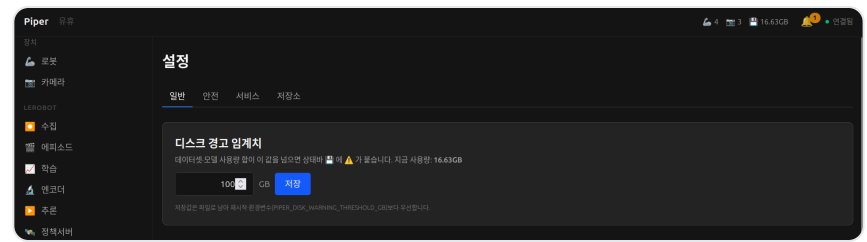


로컬 모델 — 체크포인트 목록
학습 run 아래 050000 · 045000 · … · last. 각 카드에 **추론 시작** 버튼 — 10k·20k 비교가 클릭 한 번

입출력 · 아키텍처 확인
관절 7 · 액션 7 · 카메라 top/hand 640×480 · 청크 100. 추론 전 카메라 키가 맞는지 여기서 본다



Hub 탭 — 검색 · 다운로드
smolvla_base · 팀 모델 · 데이터셋을 계정/이름으로 검색해 받기. 학습 데이터·베이스 모델 태그가 카드에

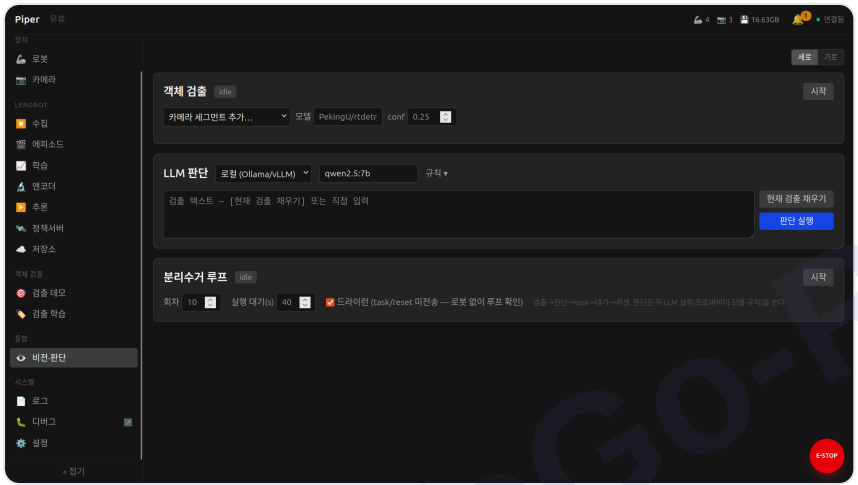


설정 — 디스크 경고 · 모델 검색 경로 · HF 토큰
빨간 경로 = "경로 없음". HF 토큰(쓰기)은 저장소 탭에서 — 호스트 로그인은 컨테이너가 못 본다

← 메뉴

검출 · 비전·판단 — YOLO → LLM → VLA (★★★ 확장)

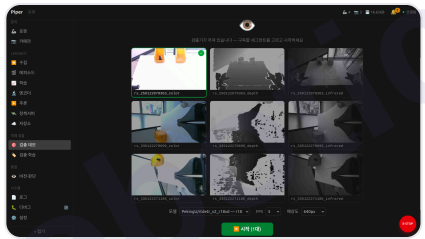
인식(YOLO) · 판단(LLM) · 실행(VLA)을 분리한 에이전트형 루프 · 실패 원인이 셋으로 갈라져 **발표 거리**가 된다



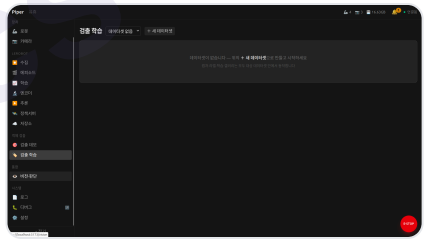
객체 검출
카메라 세그먼트 · RT-DETR 모델 · conf 임계

LLM 판단
로컬 Ollama qwen2.5:7b · 규칙("캔→파란 통") · 검출 JSON → 지시문 슬롯

분리수거 루프
회차 · 대기 · **드라이런** (로봇 없이 루프 확인) → 검출→판단→task→대기→리셋



검출 데모
세그먼트 골라 시작 — 카메라를 새로 열지 않고 녹화·추론과 같은 프레임을 구독



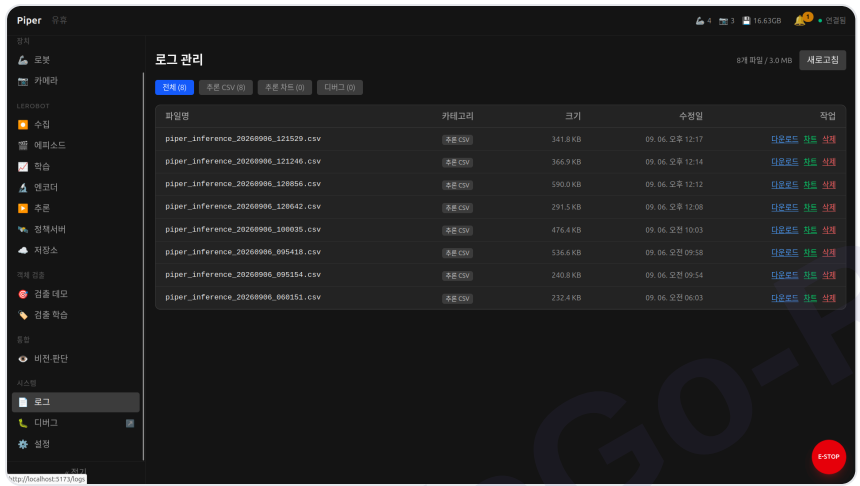
검출 학습
캡처 → 라벨 → 커스텀 YOLO 가중치. 과제 물체가 COCO에 없을 때

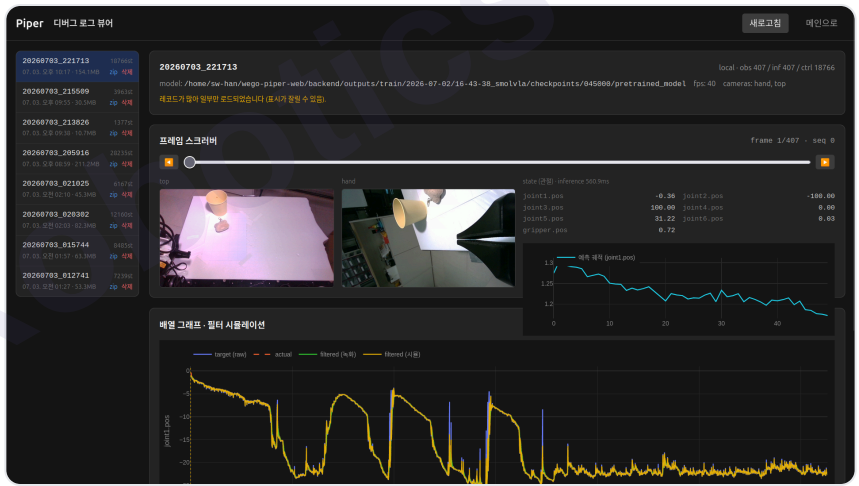
해커톤 주의 — LLM 출력은 **학습 때 쓴 task 문형과 정확히 같아야** (SmolVLA는 분포 밖 문장에 민감). 자유 작문 금지 · 7b 미만 모델은 판단이 틀림 · 먼저 드라이런 3회차 완주 후 로봇 연결

← 메뉴

로그 · 디버그 뷰어 — 실패의 증거

"왜 놓쳤지"를 감이 아니라 그래프로 · 실기 10회가 끝나면 PM이 여기부터 연다





로그 관리 — 추론 CSV 목록
실행마다 `piper_inference_날짜_시각.csv`. 다운로드 · 차트 · 삭제. 시도 번호와 시각을 기록표에 같이 적어야 나중에 짝이 맞는다

분석 도구
`tools/analyze_inference_log.py` — 관절별 |target−actual| 평균·최대 · fps·큐 그래프 PNG. "몇 번 관절이 못 따라갔나"가 숫자로

디버그 뷰어 — 프레임 스크러버
추론 때 디버그 모드로 남긴 관측(두 카메라) · 관절 state · 예측 궤적을 프레임 단위로 되감기. "정책이 그 순간 뭘 봤나"

배열 그래프 · 필터 시뮬레이션
target(raw) · actual · filtered(녹화) · filtered(시뮬) 겹쳐 보기 → 필터 값을 바꿨을 때 어땠을지 **로봇 없이** 미리 확인

읽는 순서 — ① fps·큐가 정상인가 (배포 문제) → ② target vs actual 오차가 큰 관절이 있나 (쉬프트·필터) → ③ 오차 없는데 실패했으면 그 순간 카메라 프레임 → 데이터 문제 (에피소드 페이지로)

← 메뉴

PIPER Studio · 02

OPERATION TIPS

원만한 운용을 위한 팁

도구가 못 막아주는 것 — **빛** · **팔** · **모델** · **과제**는 사람이 정한다

1. 영상 품질 — 왜곡 · 노출 · 게인 · 셔터 · 조명 · 화이트 밸런스 · 화각
2. 로봇 취약점 — 관절 쉬프트 · 발열
3. ACT vs SmolVLA — 선택 조건
4. 과제 선정 · 예시 · 시나리오 구성

이 장의 원칙을 지키면 PIPER Studio의 알람이 울릴 일이 줄어든다

← 메뉴

영상 품질 - 카메라 왜곡 현상 3가지

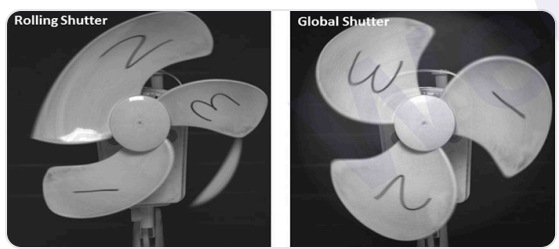
데이터 수집 시 화질을 떨어뜨리는 대표 왜곡 — 원인·증상·대응이 다르다

① 젤로 현상 Jello Effect

직선·수직 물체가 젤리처럼 기울고 출렁여 보임

원인	롤링 셔터 (행 순차 노출)
증상	형상 왜곡·기울어짐
대응	**글로벌 셔터 카메라**

저가 USB 웹캠에서 흔함



② 모션 블러 Motion Blur

피사체가 노출 중 움직여 잔상이 흐릿하게 남음

원인	긴 노출 + 빠른 물체 이동
증상	피사체 흐림·잔상
대응	**셔터 속도 ↑ · 조명 보강**

빠른 그리퍼 동작 구간 심화

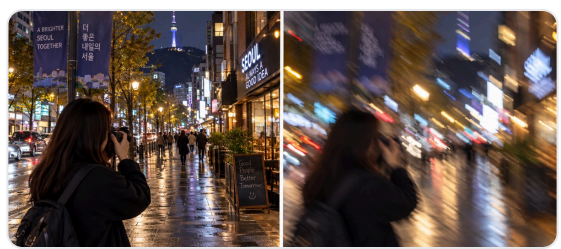


③ 핸드 블러 Hand Shake

카메라 자체 흔들림으로 화면 전체가 번짐

원인	카메라 진동·손떨림
증상	화면 전체 번짐
대응	**단단한 고정·마운트**

손목 장착 카메라 주의



GO-Robotics

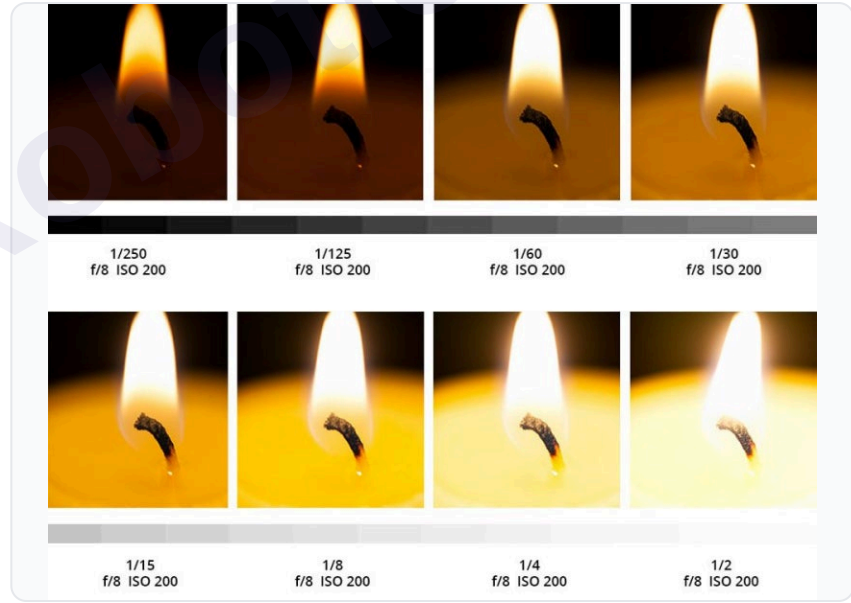
← 메뉴

영상 품질 - 카메라 노출 시간 (Exposure Time) 트레이드오프

노출 시간을 줄이면 블러가 사라져 선명해지지만, 그만큼 어두워진다



긴 노출 — 밝지만 움직이면 블러



짧은 노출 — 선명하지만 어두움

노출 ↓ 의 이득

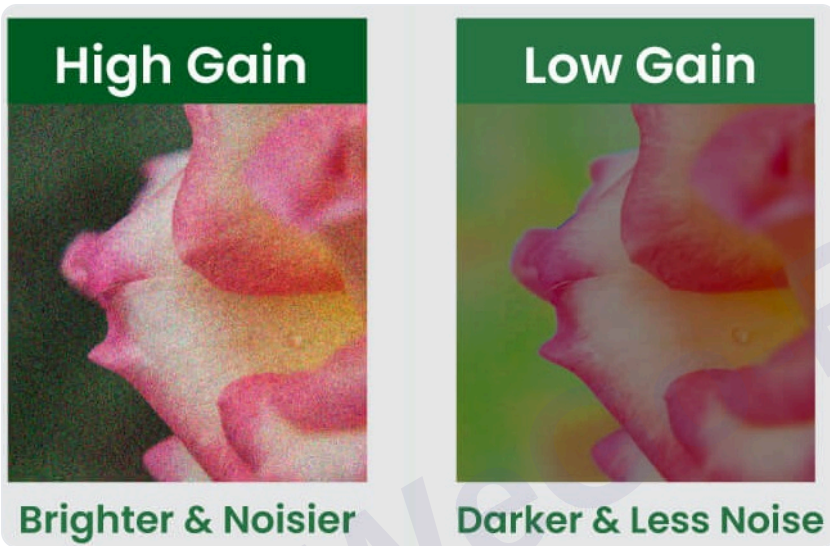
노출 ↓ 의 대가

← 메뉴

WeGo-Robotics

영상 품질 - 카메라 게인 (Gain / ISO) 트레이드오프

노출을 줄여 어두워진 사진은 게인을 올려 밝히지만, 그만큼 노이즈가 늘어난다



게인을 높일수록 밝아지지만 노이즈가 증가하는 예시

게인이란?

- 센서 신호를 **전기적으로 증폭**해 이미지를 밝힘 (ISO)
- 짧은 노출로 어두워진 사진을 **보완**하는 수단
- 신호와 함께 잡음도 증폭 → **노이즈 증가**

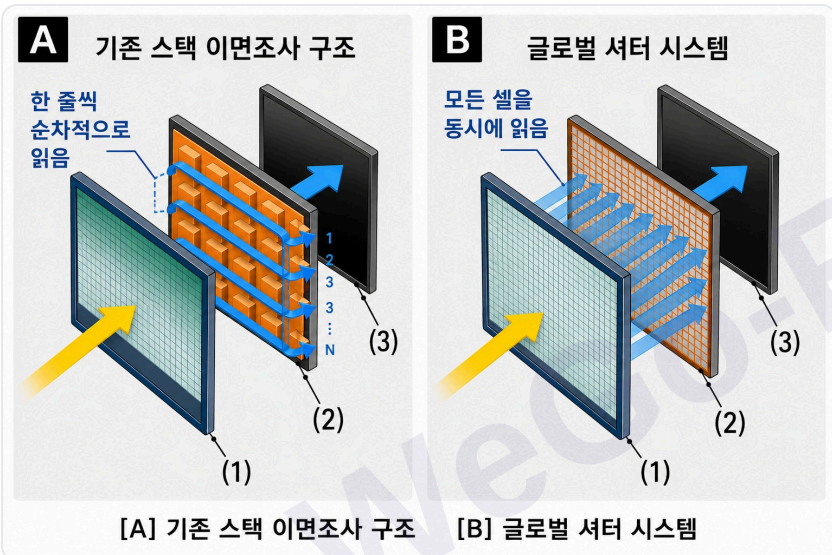
균형이 핵심

노출 ↓ (블러 제거) → 어두움 → 게인 ↑ (밝기) → 노이즈
→ **조명 보강**이 근본 해법 — 게인 의존 최소화

← 메뉴

영상 품질 - 해결책 ① 글로벌 셔터 카메라

모든 픽셀을 동시에 노출 — 젤로 현상을 근본적으로 없애는 가장 확실한 방법



[A] 기존 스택 이면조사 구조 [B] 글로벌 셔터 시스템

글로벌 셔터 산업용 카메라 예시

왜 효과적인가?

- 전체 픽셀 **동시 노출** → 행 순차 읽기의 젤로 왜곡 제거
- 빠른 동작에서도 **형상 유지** → 상태 추정 안정
- 짧은 노출과 함께 쓰면 모션 블러도 억제

항목	롤링	글로벌
노출 방식	행 순차	**동시**
젤로 왜곡	있음	**없음**
가격	저가	고가
예시	USB 웹캠	RealSense D435i · FLIR

← 메뉴

WeGo-Robotics

영상 품질 - 해결책 ② 조명 (Lighting)

밝고 균일한 조명은 짧은 노출·낮은 게인을 가능케 해 블러와 노이즈를 동시에 억제



균일한 조명으로 밝기를 확보한 촬영 환경 예시

조명이 해결하는 것

- 빛이 충분 → **짧은 노출**에도 밝기 확보 → 모션 블러 ↓
- 게인을 올릴 필요 ↓ → **노이즈 억제**
- 노출·게인 트레이드오프의 **근본 해법**

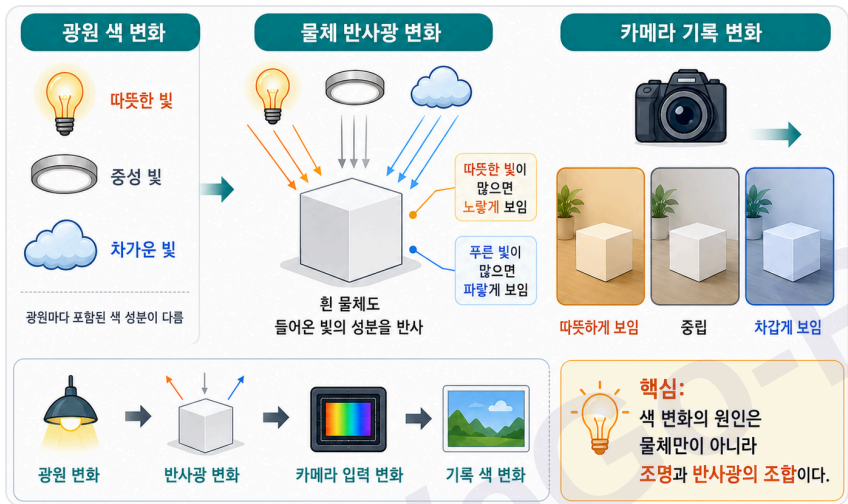
조명 구성 팁

확산광(diffuser)으로 **균일하게** — 강한 그림자·반사 방지
여러 방향 배치로 그리퍼·물체 음영 최소화

← 메뉴

영상 품질 - 화이트 밸런스

광원 색온도를 보정해 **흰색을 흰색으로** — 색 변화는 물체가 아니라 **조명·반사광**에서 온다



광원 색 변화 → 물체 반사광 변화 → 카메라 기록 색 변화

왜 중요한가 (데이터 수집)

Camera

- Auto WB → 프레임마다 색이 튄다 · 같은 장면이 다른 색으로 기록
- 정책이 **색·조명에 과적합** → 조명 바뀌면 분포 이동(distribution shift)
- 수집 때 WB ≠ 추론 때 WB → 학습-실행 색 불일치

권장 설정

- ✔ **수동(고정)** WB — 켈빈값 고정 · Auto WB / Auto Exposure 끄기
- ✔ 수집·추론 **동일 카메라 설정** 사용 (WB · 노출 · 게인)
- ✔ 조명 환경 통제 — 창문 자연광 변화·깜빡이는 형광등 주의

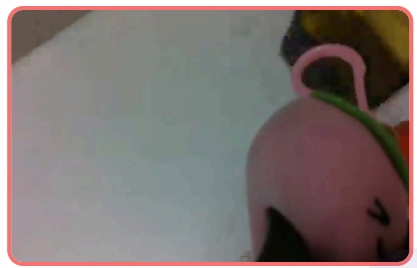
← 메뉴

영상 품질 - 카메라 화각 (FOV), 무엇이 보여야 하나?

화각이 잘못되면 정작 필요한 정보가 프레임 밖으로 — 특히 손목뷰가 까다롭다



✔ 손목뷰 — 좋은 화각



✗ 손목뷰 — 나쁜 화각



✔ 탑뷰 — 두 팔 · 물체 · 상자가 전부 프레임 안

손목뷰 화각 조건

- ✔ **그리퍼 양쪽 손가락**이 모두 프레임에 보일 것
- ✔ 집을 물체가 **그리퍼 사이(파지 지점)**로 보일 것
- ✔ 접근~파지까지 **물체가 프레임 안에 유지**
- ✔ 파지 직전 **약간의 여유 공간** 확보 (너무 클로즈업 X)

피해야 할 화각

손가락 한쪽만 보임 · 물체가 가장자리에서 잘림
너무 좁아 접근 순간 물체가 사라짐

탑뷰 화각 조건

- ✔ **에피소드 내내 로봇팔이 보일 것** — 시작 자세부터 가장 뻗은 자세까지 그리퍼가 안 잘림
- ✔ 물체 · 목표(상자) · 작업 영역 전체 + 여백 10%
- ✔ 팔이 물체를 **완전히 가리는 순간**이 없을 것 → 있으면 카메라 높이 ↑
- ✗ 여백 과다 → 물체가 작아져 특징 추출 ↓

← 메뉴

WeGo-Robotics

로봇 취약점 — 관절 쉬프트 · 발열

PIPER는 연구용 경량 협동 팔 · 산업용처럼 다루면 **관절값이 밀리고 데이터가 오염**된다 — 로봇 페이지 버전 탭에서 관절 상태를 자주 본다

can0 S-V1.8-2 slave Standby

관절	전압	드라이버 온도	모터 온도	각도 범위
joint1	23.0V	33℃	27℃	-150° ~ 150°
joint2	23.0V	32℃	27℃	0° ~ 180°
joint3	23.0V	31℃	27℃	-170° ~ 0°
joint4	23.0V	32℃	27℃	-100° ~ 100°
joint5	23.0V	33℃	27℃	-70° ~ 70°
joint6	24.0V	33℃	28℃	-180° ~ 180°

버전 탭 — 관절별 상태표
팔마다 joint1~6 · 전압 · 드라이버 온도 · 모터 온도 · 각도 범위 · 최대 속도·가속 · 토크 상태

보는 습관
수집 시작 전 · 20 ep마다 [다시 읽기] — 한 관절만 온도·전압이 다르면 그게 문제

★ 토크가 크면 관절값이 쉬프트된다

언제 — 누르며 놓기 · 걸린 채 당기기 · 무거운 물체를 팔 뻗어 들기 · 리더암 충격

증상 — 팔은 자기가 틀어진 걸 모른다. 파킹 자세가 눈에 띄게 다름 · 어제 되던 정책이 몇 cm 빗나감

대응 — 1cm 위에서 떨어뜨리기 · 저항 느끼면 즉시 멈춤 · 정렬 탭 AprilTag 검사로 확인 · 재칼리브레이션은 데이터 전체 무효

🌡 발열 예방

증상 — 2시간 연속 텔레옵 후 관절이 뜨겁고 미세 떨림 · 추종 오차 ↑

대응 — 버전 탭 **모터 온도**가 평소(27°C 전후)보다 크게 오르면 휴지 · 30분마다 5분 토크 OFF · 미사용 팔은 파킹 후 토크 OFF

← 메뉴

WeGo-Robotics

ACT vs SmolVLA — 선택 조건

기본은 ACT · 아래 조건 **전부** 만족할 때만 SmolVLA

조건	ACT	SmolVLA (smolvla_base 파인튜닝)
데이터	30~50 ep면 된다	50 ep 이상 · 변형당 10 ep 이상
언어 조건	없음 · 태스크 1개	있음 · "빨간 건 왼쪽, 파란 건 오른쪽"
GPU · **시간**	VRAM 8GB+ · 3090에서 2~3h	VRAM 24GB 권장 · 3090에서 6h+
사전학습	없음 · 처음부터	base 덕에 +26%p 수준 · 카메라 수 바뀌면 재파인튜닝
디버깅	단순	단계 많음 · 멘토 필요

ACT

★ 과제 · 단일 태스크 · GPU 1대 → **거의 모든 팀**

SmolVLA (전부 만족 시)

언어 분기 과제 · 60 ep+ · VRAM 24GB+ · 첫날 오후에 학습 시작

절충안

낮엔 ACT로 움직이는 정책 확보 → 야간에 같은 데이터로 SmolVLA → 아침에 둘 다 실기 5회

← 메뉴

WeGo-Robotics

과제 선정 원칙 — 작게, 명확하게, 리셋 쉽게

첫 1시간 안에 과제 확정 · 아래 체크리스트 전부 ✓ 아니면 재설계

좋은 과제 체크리스트

✓ 에피소드 10~30초
길수록 오차 누적 · 데이터 수집 시간 폭증

✓ 성공/실패가 한 눈에 판정
"상자 안에 들어갔다" ○ · "잘 정리됐다" ✗

✓ 허용 오차 5mm 이상 · 깔때기 기하학
PIPER + 탑뷰/손목뷰의 현실 정밀도. 기구가 오차를 흡수하는 구조(큰 상자 · 경사 포켓)가 성공률을 만든다

✓ 리셋 5초 이내
50 ep × 리셋 시간 = 수집 시간의 절반

✓ 상태 변화가 카메라에 보임
가려지는 순간이 있으면 그 구간 정책은 장님 · 물체 1~2개 · 강체 · 단색

✓ 텔레옵으로 10번 중 9번 성공
사람이 못 하면 정책도 못 한다

피해야 할 함정

✗ 여러 단계 연속 과제
"집어서 → 옮겨서 → 뚜껑 닫기" — 단계마다 성공률 곱해짐

✗ 정밀 삽입 · 조립
기어-샤프트류 — 탑뷰+손목뷰로는 구멍-축 정렬을 관측 못 한다

✗ 물체 위치 완전 랜덤
A4 용지 크기 영역 안 3×3 그리드로 제한

✗ 언어 조건 다중 태스크
"빨간 건 왼쪽, 파란 건 오른쪽" — 데이터 2배 필요. 여유 있을 때만

성공 기준 문서 (1장)
태스크 문장 · 시작 상태 사진 · 성공 상태 사진 · 물체 위치 그리드 · 시간 제한 · 실패 정의 (떨어뜨림·타임아웃·안전정지)

WeGo-Robotics

← 메뉴

과제 예시 — 난이도별

먼저 ★ 로 완주하고 시간 남으면 ★★ · ★★★ 는 데이터 수집 경험 있는 팀만

난이도	과제	에피소드	권장 ep 수	기대 성공률	핵심 난점
★	**큐브 집어 상자에 넣기** 단색 큐브 4cm · 상자 고정	10~15초	30~50	70~90%	그리퍼 닫는 타이밍 · 위치 일반화
★	**컵 뒤집어 놓기** 종이컵 · 지정 위치	10~15초	30~50	60~80%	손목 회전 궤적 일관성
★	**커피 캡슐 홀더에 투입** 홀더가 깔때기 역할	10~15초	30~50	70~90%	자기 정렬 구조 → 정밀도 요구 낮음
★★	**트레이 키팅** 부품 2종을 구획 트레이에 종류별로	15~20초	60~80	50~70%	종류 구분 · 구획 크면 정밀도 무관
★★	**색상별 분류** 빨강→왼쪽 상자 · 파랑→오른쪽 상자	15~20초	60~80	50~70%	색 기반 분기 · 데이터 균형 · SmolVLA 후보
★★★	**분리수거 (캔·페트 → 통)** 비전·판단 페이지 YOLO→LLM→VLA 루프 활용	20~30초	80~100	30~50%	외형 차이 커 인식 쉬움 · 실패 원인 3분(검출/판단/실행)
★★★	**수건 반 접기** 변형체	20~30초	100+	20~40%	상태 다양성 무한 · 해커톤 내 비권장

기대 성공률은 단일 PIPER · ACT · 카메라 2대 · 위치 그리드 제한 기준 경험치. 시나리오 원칙은 저장소의 `PiPER_AI_데모_시나리오_정리.md` · 옷 개기는 **옷 개기 세션** 참고

← 메뉴

WeGo-Robotics

시나리오 구성 유의 사항

과제가 정해졌으면 **"매번 똑같이 재현되는 한 편의 대본"**으로 만든다 — 정책은 대본 밖 상황을 모른다

고정할 것

시작·종료 상태
파킹 자세에서 출발 · 그리퍼 열림 · 물체는 그리드 칸 중앙 · 상자 위치 테이프

전략 1가지 (유니모달)
항상 같은 면을 잡고 · 같은 방향으로 돌아 · 같은 높이로 접근. "이번엔 옆으로 잡을까" 금지 — 정책이 두 전략을 평균 냄

서브태스크 순서
접근 → 하강 → 파지 → 상승 → 이동 → 하강 → 놓기 → 복귀. 순서 바꾼 에피소드 섞지 않기 (뷰어의 페이즈 트랙으로 확인)

태스크 문장
"Pick the red cube and place it in the box" · 동사 시작 · 30자 내외 · 수집 설정에 저장해 전 에피소드 동일

변화시킬 것 (계획된 범위만)

물체 위치
3×3 그리드 안에서만 · 칸 밖 배치는 일반화 시험용으로만

물체 방향
0° · 45° · 90° 세 가지 정도. 큐브면 무의미, 직육면체·컵 손잡이면 중요

복구 상황
살짝 빗나간 파지 → 재시도. 20% 비율로 **의도적으로** 포함

제외할 것 (명시적으로)

- 물체 넘어짐 · 상자 밖 낙하 후 재파지
- 사람 손이 화면에 들어오는 리셋 장면
- 방해물 · 다른 색 물체 (일반화 시험 전용)

리셋 프로토콜 (역할 고정)

1. 오퍼레이터: 팔 시작 자세 복귀 · 손 떼기
2. 셋업 담당: 다음 그리드 칸 호명 → 물체 배치 → 손 빼기
3. 셋업 담당: "고" → 수집 페이지에서 에피소드 시작
4. PM: 기록표에 칸 번호 · 결과 · 특이사항

호라이즌 점검
텔레옵 시연 시간 × 30 fps = 프레임 수. 15초면 450 프레임. ACT chunk 100이면 에피소드당 청크 4~5개 — 30초 넘으면 오차 누적 구간이 길어져 ★★★ 난이도로 봐야 함

멀티모달 제거 원리는 옷 개기 세션 · 오차 누적은 로봇 AI 기초 참고

← 메뉴

WeGo-Robotics

WHAT'S NEXT

앞으로 나아갈 방향

단일 팔 · 단일 정책으로 한 바퀴 돌았다면, 다음은 **판단이 붙은 로봇**과 **움직이는 양팔**

1. ACT · LLM · YOLO 오케스트레이션 — 인식 · 판단 · 실행을 나눈다
2. MobED와 양팔 로봇 해커톤 — 이동하며 두 팔로 일한다
3. REST API로 자체 로직 · UI — PIPER Studio를 부품처럼 쓴다

셋 다 PIPER Studio 안에 이미 자리가 있다 — 비전·판단 페이지 · 양팔 수집/추론 · 외부 제어 API

← 메뉴

방향 1 — ACT · LLM · YOLO 오케스트레이션

정책 하나가 다 하지 않는다 — **인식**(YOLO) · **판단**(LLM) · **실행**(ACT/VLA)을 나누면 실패 원인도 셋으로 갈라진다

스냅샷
팔이 빠진 순간
탑뷰 1장

→

YOLO 검출
class · conf · 위치
구조화 JSON

→

LLM 판단
규칙표 · 우선순위
"캔 → 파란 통"

→

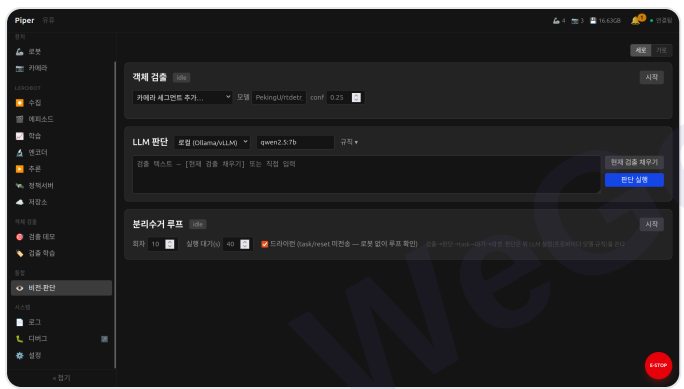
task 문장
학습 때 쓴 문형
슬롯만 채움

→

ACT / VLA 실행
30 Hz 제어 루프
LLM은 밖에

→

완료 · 리셋
다음 회차
재스냅샷



비전·판단 페이지 — 검출 · LLM · 분리수거 루프가 이미 한 화면

VLA가 정석이지만, 대회에선 무겁다
대형 VLA는 GPU·데이터·학습 시간이 30시간 예산을 넘긴다 → 인식(YOLO)·판단(LLM)·실행(ACT)을 나누면 **작은 모델 셋**으로 같은 효과

4~5인이 전원 임무를 맡는 구조
YOLO 라벨·학습 담당 / LLM 규칙·프롬프트 담당 / 텔레옵·ACT 담당 / 통합·오케스트레이터 담당 — 정책 하나면 한두 명만 바쁘고 나머지는 구경

다양하고 복잡한 미션이 가능
"캔은 파란 통, 페트는 노란 통" 같은 분기 · 여러 물체 우선순위 · 규칙 변경을 **재학습 없이** — 실행 정책은 ★ 과제 수준 그대로

주의 — LLM은 제어 루프 밖(에피소드당 1회) · 출력은 학습 때 쓴 task 문형에 슬롯만 · 먼저 드라이런 3회차 완주 후 로봇 연결

← 메뉴

방향 1 (계속) — 시퀀스 트래킹: 로봇이 지금 어느 단계인지 안다

오케스트레이터가 다음 스킬을 부르려면 **현재 단계**를 알아야 한다 — 관절 신호에서 작업 단계를 자동 라벨하고 추적하는 기능을 추가했다



에피소드 페이지 — 페이즈 트랙 + 세 신호

말단 속도
움직임 / 정지

그리퍼 지령-실측 갭
잡음 / 놓음

시작 자세 거리
출발 / 복귀

→ **상태기계**
규칙으로 단계 판정

IDLE | APPROACH | ALIGN | GRASP | HOLD | RELEASE | DONE | PARKING

수집 때 — 자동 라벨 · 편집
녹화 직후 에피소드마다 페이즈 트랙 생성 · 틀리면 [편집] · 대본 순서와 다르면 검수에서 걸러짐

학습 때 — ACT-Aux
ACT를 상속해 **행동** + **현재 단계**를 같이 내는 정책 · 라벨을 컬럼으로 구워 학습 · 기존 ACT 무수정

실행 때 — 오케스트레이터가 읽는다
GRASP에서 오래 머물면 재시도 · DONE이면 다음 스킬 호출 · 순서가 건너뛰면 정지 — 긴 미션을 단계 단위로 감시

해커톤에서 — 라벨·뷰어는 바로 사용 · ACT-Aux는 기본 ACT가 되는 팀만 · 실패 코드(F1 접근 · F2 파지 · F4 정지)를 페이즈로 자동 분류 가능

← 메뉴

방향 2 — MobED와 양팔 로봇 해커톤

고정 테이블에서 벗어난다 — **이동 플랫폼 위의 양팔**로 "가서 · 잡고 · 옮기는" 과제



MobED + 양팔 · 손목·헤드 카메라 탑재 구성

MobED — 현대차그룹 플랫폼
DnL 모듈: 바퀴마다 구동·조향·승강 → 경사·턱에서도 상판 수평 · 2025 IREX 양산형 · 위고로보틱스 얼라이언스 참여

양팔 — PIPER Studio에 이미
수집 페이지 단팔/양팔 전환 · 팔 4대 · 14축 액션 · left_top / right_hand 키 · 수집·추론·파킹 한 구현

🏭 대회 수준을 산업 현장 시나리오로

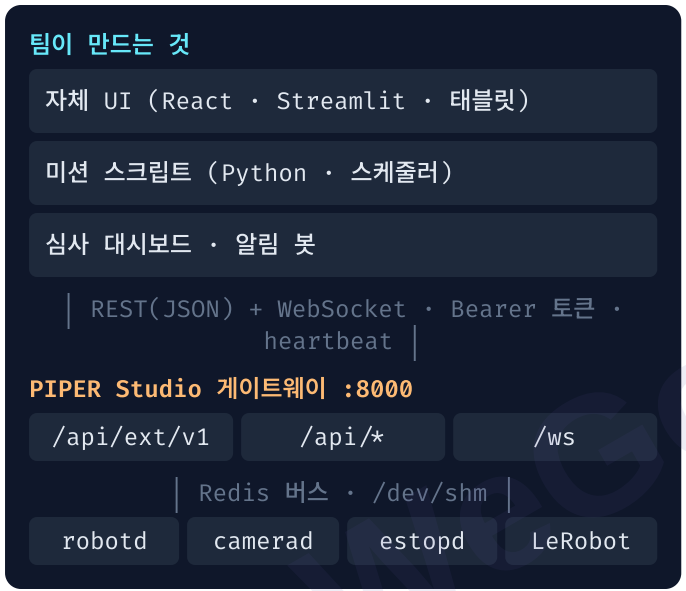
단계	시나리오	현장 스토리 — 전부 이동 포함
이동	**박스 이동**	물류 — 선반의 박스를 두 팔로 들어 MobED로 옮겨 작업대에
이동	**배전반 순찰 · 과열 감시**	설비 — 배전반까지 이동 → 문 열기 → 열화상·카메라로 점검 → 문 닫기

← 메뉴

WeGo-Robotics

방향 3 — REST API로 자체 로직 · UI 구현

화면을 쓰는 대신 **부품으로 부른다** — 카메라·로봇·녹화·학습·추론·검출이 전부 HTTP/WebSocket 엔드포인트



장치 수준은 일부러 안 열림 — 등록·칼리브·프로파일은 화면에서, **실행·감시·판단은 API로**

엔드포인트	할 수 있는 것
`/api/ext/v1` 미션 · 상태	분리수거 같은 **미션 제출·감시·취소** · 종합 상태 · yolod 제어 · E-stop
`/api/models/inference`	체크포인트 골라 추론 시작·정지 · `/api/params`로 실시간 파라미터
`/api/datasets` · `/api/hub`	데이터셋 목록·편집 · Hub 검색·다운로드·업로드
`/api/vision` · `/api/yolo`	YOLO 검출 결과 · LLM 판단 테스트 · 커스텀 YOLO 학습
`/api/eval` · `/api/logs`	시도별 성공/실패 기록 · 통계 · 추론 CSV 다운로드
`WS /ws`	로그 · 프로세스 상태 · 경보 실시간 스트림

해커톤에서 만들 수 있는 것
태블릿용 "시작·리셋·성공/실패" 한 화면 · 10회 자동 평가 스크립트 · 배전반 순찰 스케줄러 · 결과 슬랙 봇

심사·운영 쪽도
전 팀 상태·성공률을 한 대시보드에 · 시도 기록이 API로 남아 채점 자동화

안전 계약은 그대로 — 외부 클라이언트가 heartbeat를 보내야 하고, 끊기면 estopd가 2.5초 안에 세운다 · `PIPER_API_TOKEN` 미설정이면 전체 503 · 켜는 것이 명시적 행위

← 메뉴

WeGo-Robotics

PIPER Studio · 로봇 AI 해커톤

QUESTIONS

Q & A

배우는 데 쓰던 시간이 **데이터 모으는 시간**이 되도록 — 궁금한 점 편하게 질문해 주세요

홈페이지
wego-robotics.com

커뮤니티 (Discourse)
wego-robotics.discourse.group

PIPER Studio
github.com/WeGo-Robotics/wego-piper-web

문의
한승욱 책임 연구원 · sw.han@wego-robotics.com

감사합니다

← 메뉴

ⓒ 2026 WeGo-Robotics. 본 자료의 무단 복사·배포·유출은 법적으로 보호받지 못하며, 관련 법령에 따라 책임이 따를 수 있습니다.